Biostat 212a Homework 6

Due Mar 22, 2025 @ 11:59PM

Nathaniel Boyle and 006525153

2025-03-21

Table of contents

Load R libraries.

```
if (!requireNamespace("pheatmap", quietly = TRUE)) {
  install.packages("pheatmap", repos = "https://cloud.r-project.org/")
}
if (!requireNamespace("ggdendro", quietly = TRUE)) {
  install.packages("ggdendro", repos = "https://cloud.r-project.org/")
}
if (!requireNamespace("tidyclust", quietly = TRUE)) {
  install.packages("tidyclust", repos = "https://cloud.r-project.org/")
}
if (!requireNamespace("RcppHungarian", quietly = TRUE)) {
  install.packages("RcppHungarian", repos = "https://cloud.r-project.org/")
}
if (!requireNamespace("glmnet", quietly = TRUE)) {
  install.packages("glmnet", repos = "https://cloud.r-project.org/")
```

```
}
if (!requireNamespace("dplyr", quietly = TRUE)) {
  install.packages("dplyr", repos = "https://cloud.r-project.org/")
}
if (!requireNamespace("FactoMineR", quietly = TRUE)) {
  install.packages("FactoMineR", repos = "https://cloud.r-project.org/")
}
if (!requireNamespace("factoextra", quietly = TRUE)) {
  install.packages("factoextra", repos = "https://cloud.r-project.org/")
}
# Core libraries for data manipulation and visualization
library(tidyverse)  # for data manipulation and visualization
library(tidymodels) # A framework for building machine learning models
library(readr)      # Functions for reading and writing data
library(ggplot2)    # for creating visualizations
library(ggdendro)   # visualize hierarchical clustering results using dendrograms

# Machine learning and statistical modeling libraries
library(glmnet)     # Implements LASSO and Ridge regression
library(yardstick)  # evaluating machine learning models
library(workflows)  # Facilitates building and managing workflows for machine learning tasks
library(parsnip)    # Part of `tidymodels` for specifying and fitting models
library(randomForest) # Implements the random forest algorithm
library(xgboost)    # Implements the gradient boosting machine (GBM) algorithm
library(caret)      # for training and evaluating machine learning models

# Time series and clustering libraries
library(tswge)      # For time series analysis
library(tidyclust)  # Provides tools for clustering tasks
library(RcppHungarian) # Implements the Hungarian algorithm for solving problems
library(dplyr)      # For data manipulation,

# Data transformation and visualization utilities
library(reshape2)   # Provides functions for reshaping data (e.g., melting and casting)
library(gridExtra)  # Facilitates arranging multiple plots in a grid layout
library(pheatmap)   # For creating heatmaps with advanced customization options

# Dimensionality reduction and visualization libraries
library(umap)       # Implements the UMAP
library(FactoMineR) # A package for multivariate data analysis (e.g., PCA)
library(factoextra) # Functions for visualizing the results of PCA
```

0.1 New York Stock Exchange (NYSE) data (1962-1986) (140 pts)

Historical trading statistics from the New York Stock Exchange. Daily values of the normalized log trading volume, DJIA return, and log volatility are shown for a 24-year period from 1962-1986. We wish to predict trading volume on any day, given the history on all earlier days. To the left of the red bar (January 2, 1980) is training data, and to the right test data.

The NYSE.csv file contains three daily time series from the New York Stock Exchange (NYSE) for the period Dec 3, 1962-Dec 31, 1986 (6,051 trading days).

- `Log trading volume` (v_t): This is the fraction of all outstanding shares that are traded on that day, relative to a 100-day moving average of past turnover, on the log scale.
- `Dow Jones return` (r_t): This is the difference between the log of the Dow Jones Industrial Index on consecutive trading days.
- `Log volatility` (z_t): This is based on the absolute values of daily price movements.

```
# Read in NYSE data from url

url = "https://raw.githubusercontent.com/ucla-biostat-212a/2025winter/master/slides/data/NYSE
NYSE <- read_csv(url)

NYSE
```

```
# A tibble: 6,051 x 6
   date       day_of_week DJ_return log_volume log_volatility train
   <date>     <chr>           <dbl>      <dbl>          <dbl> <lgl>
 1 1962-12-03 mon         -0.00446     0.0326           -13.1 TRUE
 2 1962-12-04 tues         0.00781     0.346            -11.7 TRUE
 3 1962-12-05 wed          0.00384     0.525            -11.7 TRUE
 4 1962-12-06 thur        -0.00346     0.210            -11.6 TRUE
 5 1962-12-07 fri          0.000568    0.0442           -11.7 TRUE
 6 1962-12-10 mon         -0.0108      0.133            -10.9 TRUE
 7 1962-12-11 tues         0.000124   -0.0115           -11.0 TRUE
 8 1962-12-12 wed          0.00336     0.00161          -11.0 TRUE
 9 1962-12-13 thur        -0.00330    -0.106            -11.0 TRUE
10 1962-12-14 fri          0.00447    -0.138            -11.0 TRUE
# i 6,041 more rows
```

```
acfdf <- function(vec) {
  vacf <- acf(vec, plot = F)
  with(vacf, data.frame(lag, acf))
```

```
}

ggacf <- function(vec) {
  ac <- acfdf(vec)
  ggplot(data = ac, aes(x=lag, y = acf)) +geom_hline(aes(yintercept = 0)) +
    geom_segment(mapping = aes(xend = lag, yend = 0))
}

tplot <- function(vec) {
  df <- data.frame(X = vec, t = seq_along(vec))
  ggplot(data = df, aes(x = t, y = X)) +geom_line()
}
```

The **autocorrelation** at lag ℓ is the correlation of all pairs $(v_t, v_{t-\ell})$ that are ℓ trading days apart. These sizable correlations give us confidence that past values will be helpful in predicting the future.

```
ggacf(NYSE$log_volume) + ggthemes::theme_few()
```

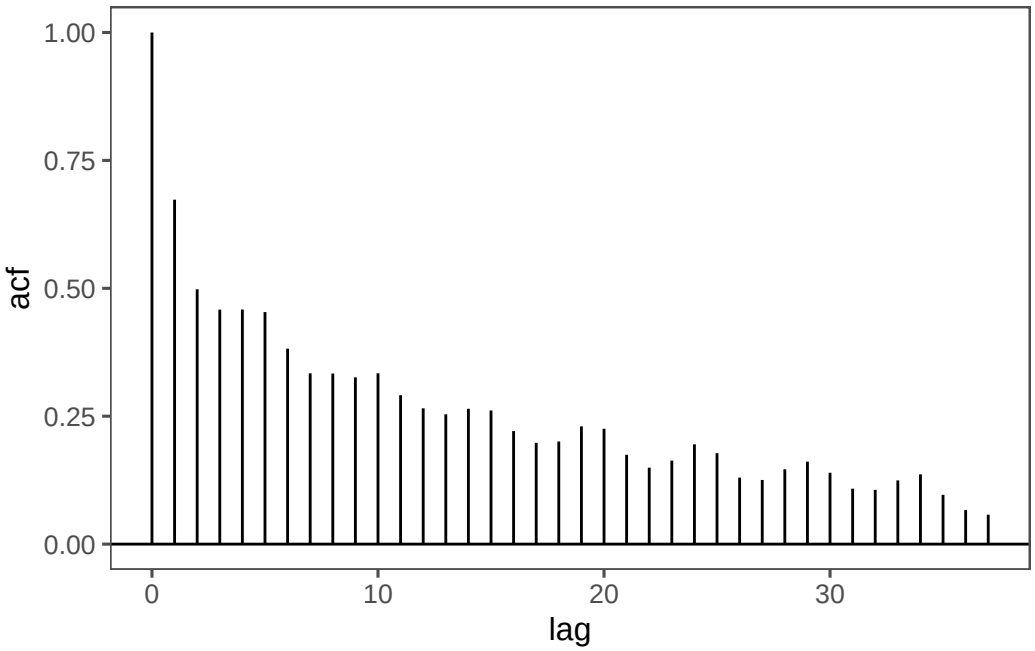
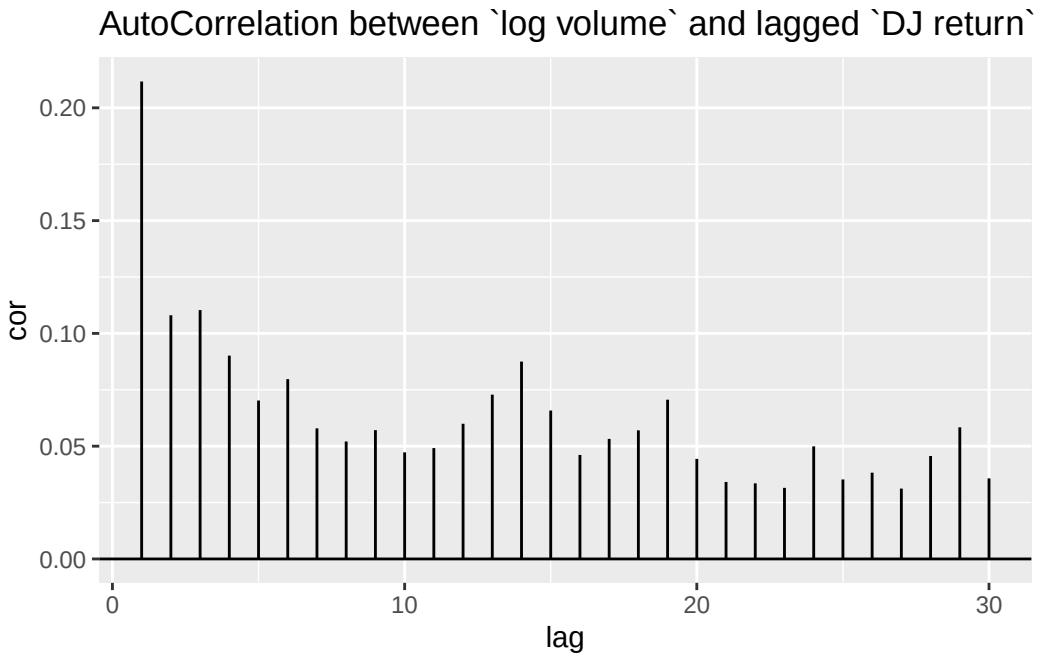


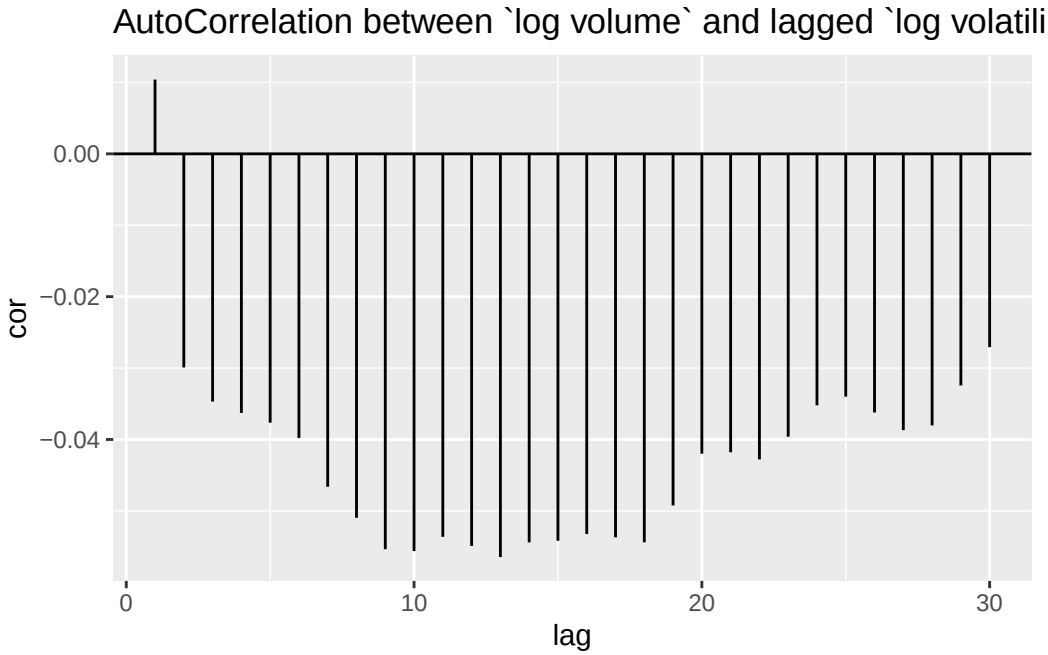
Figure 1: The autocorrelation function for log volume. We see that nearby values are fairly strongly correlated, with correlations above 0.2 as far as 20 days apart.

Do a similar plot for (1) the correlation between v_t and lag ℓ `Dow Jones return` $r_{t-\ell}$ and (2) correlation between v_t and lag ℓ `Log volatility` $z_{t-\ell}$.

```
seq(1, 30) %>%
  map(function(x) {cor(NYSE$log_volume , lag(NYSE$DJ_return, x),
                       use = "pairwise.complete.obs")}) %>%
  unlist() %>%
  tibble(lag = 1:30, cor = .) %>%
  ggplot(aes(x = lag, y = cor)) +
  geom_hline(aes(yintercept = 0)) +
  geom_segment(mapping = aes(xend = lag, yend = 0)) +
  ggtitle("AutoCorrelation between `log volume` and lagged `DJ return`")
```




```
seq(1, 30) %>%
  map(function(x) {cor(NYSE$log_volume , lag(NYSE$log_volatility, x),
                       use = "pairwise.complete.obs")}) %>%
  unlist() %>%
  tibble(lag = 1:30, cor = .) %>%
  ggplot(aes(x = lag, y = cor)) +
  geom_hline(aes(yintercept = 0)) +
  geom_segment(mapping = aes(xend = lag, yend = 0)) +
  ggtitle("AutoCorrelation between `log volume` and lagged `log volatility`")
```



0.1.1 Project goal

Our goal is to forecast daily `Log trading volume`, using various machine learning algorithms we learnt in this class.

The data set is already split into train (before Jan 1st, 1980, $n_{\text{train}} = 4,281$) and test (after Jan 1st, 1980, $n_{\text{test}} = 1,770$) sets.

In general, we will tune the lag L to achieve best forecasting performance. In this project, we would fix $L = 5$. That is we always use the previous five trading days' data to forecast today's `log trading volume`.

Pay attention to the nuance of splitting time series data for cross validation. Study and use the `time-series` functionality in tidymodels. Make sure to use the same splits when tuning different machine learning algorithms.

Use the R^2 between forecast and actual values as the cross validation and test evaluation criterion.

0.1.2 Baseline method (20 pts)

We use the straw man (use yesterday's value of `log trading volume` to predict that of today) as the baseline method. Evaluate the R^2 of this method on the test data.

```r
#baseline method

# Create the baseline prediction (using the previous day's log trading volume)
NYSE_baseline <- NYSE |>
  mutate(BaselinePrediction = lag(log_volume, n = 1)) |>
  filter(!is.na(BaselinePrediction))

# View the first few rows to check the baseline predictions
head(NYSE_baseline)
```

```
# A tibble: 6 x 7
  date       day_of_week DJ_return log_volume log_volatility train
  <date>     <chr>           <dbl>      <dbl>          <dbl> <lgl>
1 1962-12-04 tues          0.00781      0.346          -11.7 TRUE
2 1962-12-05 wed           0.00384      0.525          -11.7 TRUE
3 1962-12-06 thur         -0.00346      0.210          -11.6 TRUE
4 1962-12-07 fri           0.000568     0.0442         -11.7 TRUE
5 1962-12-10 mon          -0.0108       0.133          -10.9 TRUE
6 1962-12-11 tues          0.000124    -0.0115         -11.0 TRUE
# i 1 more variable: BaselinePrediction <dbl>
```

```r
# Check the structure of the data to ensure that the Date column is correctly parsed
str(NYSE_baseline)
```

```
tibble [6,050 x 7] (S3: tbl_df/tbl/data.frame)
 $ date              : Date[1:6050], format: "1962-12-04" "1962-12-05" ...
 $ day_of_week       : chr [1:6050] "tues" "wed" "thur" "fri" ...
 $ DJ_return         : num [1:6050] 0.007813 0.003845 -0.003462 0.000568 -0.010824 ...
 $ log_volume        : num [1:6050] 0.3462 0.5253 0.2102 0.0442 0.1332 ...
 $ log_volatility    : num [1:6050] -11.7 -11.7 -11.6 -11.7 -10.9 ...
 $ train             : logi [1:6050] TRUE TRUE TRUE TRUE TRUE TRUE ...
 $ BaselinePrediction: num [1:6050] 0.0326 0.3462 0.5253 0.2102 0.0442 ...
```

```r
head(NYSE_baseline)
```

```
# A tibble: 6 x 7
  date       day_of_week DJ_return log_volume log_volatility train
  <date>     <chr>           <dbl>      <dbl>          <dbl> <lgl>
1 1962-12-04 tues          0.00781      0.346          -11.7 TRUE
2 1962-12-05 wed           0.00384      0.525          -11.7 TRUE
```

```
3 1962-12-06 thur        -0.00346      0.210           -11.6 TRUE
4 1962-12-07 fri          0.000568     0.0442          -11.7 TRUE
5 1962-12-10 mon         -0.0108       0.133           -10.9 TRUE
6 1962-12-11 tues         0.000124    -0.0115          -11.0 TRUE
# i 1 more variable: BaselinePrediction <dbl>
```

```
# Convert 'Date' column to Date type (if not already)
NYSE_baseline$Date <- as.Date(NYSE_baseline$date, format = "%m/%d/%Y")

# Ensure that the 'log_volume' column exists and is named correctly
head(NYSE_baseline)
```

```
# A tibble: 6 x 8
  date       day_of_week DJ_return log_volume log_volatility train
  <date>     <chr>           <dbl>      <dbl>          <dbl> <lgl>
1 1962-12-04 tues          0.00781     0.346          -11.7 TRUE
2 1962-12-05 wed           0.00384     0.525          -11.7 TRUE
3 1962-12-06 thur         -0.00346     0.210          -11.6 TRUE
4 1962-12-07 fri           0.000568    0.0442         -11.7 TRUE
5 1962-12-10 mon          -0.0108      0.133          -10.9 TRUE
6 1962-12-11 tues          0.000124   -0.0115         -11.0 TRUE
# i 2 more variables: BaselinePrediction <dbl>, Date <date>
```

```
# Create the baseline prediction (using the previous day's log trading volume)
NYSE_baseline <- NYSE_baseline |>
  mutate(BaselinePrediction = lag(log_volume, n = 1)) |>
  filter(!is.na(BaselinePrediction))

# View the first few rows to check the baseline predictions
head(NYSE_baseline)
```

```
# A tibble: 6 x 8
  date       day_of_week DJ_return log_volume log_volatility train
  <date>     <chr>           <dbl>      <dbl>          <dbl> <lgl>
1 1962-12-05 wed           0.00384     0.525          -11.7 TRUE
2 1962-12-06 thur         -0.00346     0.210          -11.6 TRUE
3 1962-12-07 fri           0.000568    0.0442         -11.7 TRUE
4 1962-12-10 mon          -0.0108      0.133          -10.9 TRUE
5 1962-12-11 tues          0.000124   -0.0115         -11.0 TRUE
6 1962-12-12 wed           0.00336     0.00161        -11.0 TRUE
# i 2 more variables: BaselinePrediction <dbl>, Date <date>
```

```r
# Split the data into training and test sets (before and after January 1st, 1980)
train_data <- NYSE_baseline %>%
  filter(Date < "1980-01-01")

test_data <- NYSE_baseline %>%
  filter(Date >= "1980-01-01")

# View the train and test set sizes to ensure proper split
dim(train_data)
```

```
[1] 4279    8
```

```r
dim(test_data)
```

```
[1] 1770    8
```

```r
# Calculate the R^2 for the baseline model on the test data
# Use lm (linear model) to calculate R^2 between predicted and actual log trading volume
model <- lm(log_volume ~ BaselinePrediction, data = test_data)

# Get the R^2 value
summary(model)$r.squared
```

```
[1] 0.348125
```

The test R^2 value of this baseline model is 0.35.

0.1.3 Autoregression (AR) forecaster (30 pts)

- Let

$$y = \begin{pmatrix} v_{L+1} \\ v_{L+2} \\ v_{L+3} \\ \vdots \\ v_T \end{pmatrix}, \quad M = \begin{pmatrix} 1 & v_L & v_{L-1} & \cdots & v_1 \\ 1 & v_{L+1} & v_L & \cdots & v_2 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & v_{T-1} & v_{T-2} & \cdots & v_{T-L} \end{pmatrix}.$$

- Fit an ordinary least squares (OLS) regression of y on M, giving

$$\hat{v}_t = \hat{\beta}_0 + \hat{\beta}_1 v_{t-1} + \hat{\beta}_2 v_{t-2} + \cdots + \hat{\beta}_L v_{t-L},$$

  known as an **order-L autoregression** model or **AR(L)**.

```r
# Example time series data
log_volume <- NYSE$log_volume

# Define the order of the autoregression (L=3 for AR(3) model)
L <- 3

# Create the matrix M with lagged values (first column is for the intercept term)
T <- length(log_volume)
M <- cbind(
  1,  # intercept term (constant)
  lag(log_volume, 1)[(L+1):T],  # lag 1
  lag(log_volume, 2)[(L+1):T],  # lag 2
  lag(log_volume, 3)[(L+1):T]   # lag 3
)

# Create the vector y (the target variable, which is log_volume from L+1 to T)
y <- log_volume[(L+1):T]

# Fit the OLS model using the lm() function (exclude intercept in the formula
# since it's already included in M)
model <- lm(y ~ M - 1)  # '-1' removes the intercept term (since it's already in M)

# View the model coefficients
summary(model)
```

```
Call:
lm(formula = y ~ M - 1)

Residuals:
     Min       1Q   Median       3Q      Max
-1.23068 -0.10214 -0.00333  0.10026  1.03631

Coefficients:
    Estimate Std. Error t value Pr(>|t|)
M1 -0.002227   0.002181  -1.021   0.3073
M2  0.602692   0.012659  47.609   <2e-16 ***
M3 -0.025663   0.014841  -1.729   0.0838 .
M4  0.175733   0.012659  13.883   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 0.1695 on 6044 degrees of freedom
Multiple R-squared:  0.4744,    Adjusted R-squared:  0.474
F-statistic:  1364 on 4 and 6044 DF,  p-value: < 2.2e-16
```

R^2 multiple is 0.47. Adjusted R^2 is 0.47.

- Tune AR(5) with elastic net (lasso + ridge) regularization using all 3 features on the training data, and evaluate the test performance.

```r
# Convert Date column to Date type if it's not already
NYSE$date <- as.Date(NYSE$date, format = "%m/%d/%Y")

# Create lagged features for log_volume, DJIA_return, and log_volatility
L <- 5  # We will use AR(5) model, so L = 5

NYSE_lagged <- NYSE %>%
  mutate(
    lag_log_volume_1 = lag(log_volume, 1),
    lag_log_volume_2 = lag(log_volume, 2),
    lag_log_volume_3 = lag(log_volume, 3),
    lag_log_volume_4 = lag(log_volume, 4),
    lag_log_volume_5 = lag(log_volume, 5),

    lag_DJ_return_1 = lag(DJ_return, 1),
    lag_DJ_return_2 = lag(DJ_return, 2),
    lag_DJ_return_3 = lag(DJ_return, 3),
    lag_DJ_return_4 = lag(DJ_return, 4),
    lag_DJ_return_5 = lag(DJ_return, 5),

    lag_log_volatility_1 = lag(log_volatility, 1),
    lag_log_volatility_2 = lag(log_volatility, 2),
    lag_log_volatility_3 = lag(log_volatility, 3),
    lag_log_volatility_4 = lag(log_volatility, 4),
    lag_log_volatility_5 = lag(log_volatility, 5)
  ) %>%
  filter(complete.cases(.))  # Remove rows with NA values caused by lagging

# Split data into training (before 1980) and test (after 1980)
train_data <- NYSE_lagged %>% filter(date < "1980-01-01")
test_data <- NYSE_lagged %>% filter(date >= "1980-01-01")

# Inspect the sizes of the train and test sets
dim(train_data)
```

```
[1] 4276   21
```

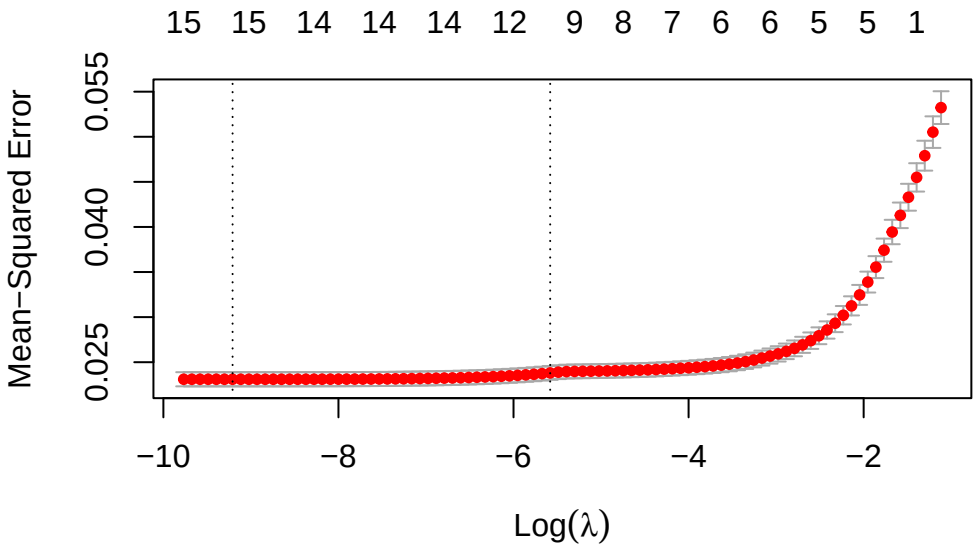
```
dim(test_data)
```

```
[1] 1770   21
```

```
# Prepare training data
X_train <- as.matrix(train_data %>% select(starts_with("lag_")))  # All lagged features
y_train <- train_data$log_volume  # Target variable

# Prepare test data
X_test <- as.matrix(test_data %>% select(starts_with("lag_")))
y_test <- test_data$log_volume


# Fit the Elastic Net model using cross-validation
cv_model <- cv.glmnet(X_train, y_train, alpha = 0.5, nfolds = 10)  # alpha = 0.5 for Elastic

# View the cross-validation results
cv_model
```

```
Call:  cv.glmnet(x = X_train, y = y_train, nfolds = 10, alpha = 0.5)

Measure: Mean-Squared Error

      Lambda Index Measure        SE Nonzero
min 0.000100    88 0.02311 0.0007841      15
1se 0.003768    49 0.02381 0.0007691      11
```

```
# Plot the cross-validation curve
plot(cv_model)
```




```
# Extract the cross-validation R-squared values
cv_r_squared <- 1 - cv_model$cvm / mean((y_train - mean(y_train))^2)

# Print the max cv R-squared value
print(max(cv_r_squared))
```

```
[1] 0.5663301
```

```
# Predict on the test set
y_pred <- predict(cv_model, X_test, s = "lambda.min")

# Calculate R^2 on the test set
rss <- sum((y_test - y_pred)^2)
tss <- sum((y_test - mean(y_test))^2)
r_squared <- 1 - (rss / tss)

# Print the test R^2 value
r_squared
```

```
[1] 0.4128989
```

The CV R^2 value of the baseline model is 0.57 The test R^2 value of this model is 0.41

- Hint: Workflow: Lasso is a good starting point.

0.1.4 Random forest forecaster (30pts)

- Use the same features as in AR(L) for the random forest. Tune the random forest and evaluate the test performance.

```
# Create lagged features for log_volume, DJIA_return, and log_volatility
L <- 5  # We will use AR(5) model, so L = 5

# Suppress deprecated warning
options(warn = -1)  # Suppress all warnings

NYSE_lagged <- NYSE %>%
  mutate(
    lag_log_volume_1 = lag(log_volume, 1),
    lag_log_volume_2 = lag(log_volume, 2),
    lag_log_volume_3 = lag(log_volume, 3),
    lag_log_volume_4 = lag(log_volume, 4),
    lag_log_volume_5 = lag(log_volume, 5),

    lag_DJ_return_1 = lag(DJ_return, 1),
    lag_DJ_return_2 = lag(DJ_return, 2),
    lag_DJ_return_3 = lag(DJ_return, 3),
    lag_DJ_return_4 = lag(DJ_return, 4),
    lag_DJ_return_5 = lag(DJ_return, 5),

    lag_log_volatility_1 = lag(log_volatility, 1),
    lag_log_volatility_2 = lag(log_volatility, 2),
    lag_log_volatility_3 = lag(log_volatility, 3),
    lag_log_volatility_4 = lag(log_volatility, 4),
    lag_log_volatility_5 = lag(log_volatility, 5)
  ) %>%
  filter(complete.cases(.))  # Remove rows with NA values caused by lagging

# Split data into training (before 1980) and test (after 1980)
train_data <- NYSE_lagged %>% filter(date < "1980-01-01")
test_data <- NYSE_lagged %>% filter(date >= "1980-01-01")

# Inspect the sizes of the train and test sets
dim(train_data)
```

```
[1] 4276   21
```

```
dim(test_data)
```

```
[1] 1770   21
```

```
# Prepare the training data matrix and target variable
X_train <- train_data %>% select(starts_with("lag_"))
y_train <- train_data$log_volume

# Prepare the test data matrix and target variable
X_test <- test_data %>% select(starts_with("lag_"))
y_test <- test_data$log_volume

# Set up a training control object for tuning (use 10-fold cross-validation)
train_control <- trainControl(method = "cv", number = 10)

# Correct the tuning grid: Only include mtry for Random Forest
tune_grid <- expand.grid(
  mtry = c(2, 4, 6, 8, 10)  # Number of variables to consider at each split
)

# Tune the random forest model
rf_tune <- train(
  x = X_train,
  y = y_train,
  method = "rf", # Random Forest method
  trControl = train_control,
  tuneGrid = tune_grid,
  ntree = 500  # Number of trees in the forest
)

# View the best tuned parameters
print(rf_tune$bestTune)
```

```
  mtry
3    6
```

```
# Print the best model's cross-validation R-squared
best_r_squared <- max(rf_tune$results$Rsquared)
cat("Best Cross-validation R-squared for Random Forest: ", best_r_squared, "\n")
```

```
Best Cross-validation R-squared for Random Forest:  0.5755052
```

```r
# turn warnings back on
options(warn = 0)

# our best forest model was with mtry 6
# this means 6 variables considered each split provided
# the best outcome
```

```r
#evaluate model performance

# Make predictions on the test set
y_pred_rf <- predict(rf_tune, X_test)

# Calculate R^2 on the test set
rss_rf <- sum((y_test - y_pred_rf)^2)
tss_rf <- sum((y_test - mean(y_test))^2)
r_squared_rf <- 1 - (rss_rf / tss_rf)

# Print the R^2 value
print(r_squared_rf)
```

```
[1] 0.4039408
```

```r
#found R^2 is 0.4, lower than previous models.
```

The test R^2 of the Random forest is 0.40

- Hint: Workflow: Random Forest for Prediction is a good starting point.

0.1.5 Boosting forecaster (30pts)

- Use the same features as in AR(L) for the boosting. Tune the boosting algorithm and evaluate the test performance.

```r
# Create lagged features for log_volume, DJIA_return, and log_volatility
L <- 5  # We will use AR(5) model, so L = 5

NYSE_lagged <- NYSE %>%
  mutate(
```

```
    lag_log_volume_1 = lag(log_volume, 1),
    lag_log_volume_2 = lag(log_volume, 2),
    lag_log_volume_3 = lag(log_volume, 3),
    lag_log_volume_4 = lag(log_volume, 4),
    lag_log_volume_5 = lag(log_volume, 5),

    lag_DJ_return_1 = lag(DJ_return, 1),
    lag_DJ_return_2 = lag(DJ_return, 2),
    lag_DJ_return_3 = lag(DJ_return, 3),
    lag_DJ_return_4 = lag(DJ_return, 4),
    lag_DJ_return_5 = lag(DJ_return, 5),

    lag_log_volatility_1 = lag(log_volatility, 1),
    lag_log_volatility_2 = lag(log_volatility, 2),
    lag_log_volatility_3 = lag(log_volatility, 3),
    lag_log_volatility_4 = lag(log_volatility, 4),
    lag_log_volatility_5 = lag(log_volatility, 5)
  ) %>%
  filter(complete.cases(.))  # Remove rows with NA values caused by lagging

# Split data into training (before 1980) and test (after 1980)
train_data <- NYSE_lagged %>% filter(date < "1980-01-01")
test_data <- NYSE_lagged %>% filter(date >= "1980-01-01")

# Prepare the training data matrix and target variable
X_train <- train_data %>% select(starts_with("lag_"))
y_train <- train_data$log_volume

# Prepare the test data matrix and target variable
X_test <- test_data %>% select(starts_with("lag_"))
y_test <- test_data$log_volume

# Convert the data to matrix format for xgboost
X_train_matrix <- as.matrix(X_train)
X_test_matrix <- as.matrix(X_test)

# Set up a training control object for tuning (use 5-fold cross-validation)
train_control <- trainControl(method = "cv", number = 5)

# Set up the correct tuning grid
tune_grid <- expand.grid(
  nrounds = c(10, 50, 100),           # boosting iterations
```

```r
  max_depth = c(3, 5),                 # Limit max_depth
  eta = c(0.01, 0.1),                  # Use smaller eta values
  gamma = c(0, 0.1),                   # Reduce gamma values
  colsample_bytree = c(0.7),           # Keep colsample_bytree constant
  subsample = c(0.7),                  # Keep subsample constant
  min_child_weight = c(1)              # Reduce min_child_weight values
)


# Train the xgboost model with tuning
xgb_tune <- train(
  x = X_train_matrix,
  y = y_train,
  method = "xgbTree",  # Use xgboost algorithm
  trControl = train_control,
  tuneGrid = tune_grid
)
```

```
[14:39:34] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:34] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:34] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:34] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:35] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:35] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:35] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:35] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:35] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:35] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:35] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:35] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:36] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:36] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:36] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:36] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:37] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:37] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:37] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:37] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:37] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:37] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:38] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:38] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
```

```
[14:39:38] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:38] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:38] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:38] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:39] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:39] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:39] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:39] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:39] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:39] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:39] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:39] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:40] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:40] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:40] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:40] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:40] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:40] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:41] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:41] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:41] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:41] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:41] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:41] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:42] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:42] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:42] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:42] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:42] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:42] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:43] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:43] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:43] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:43] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:43] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:43] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:44] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:44] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:44] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:44] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:44] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:44] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:44] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
```

```
[14:39:44] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:45] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:45] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:45] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:45] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:45] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:45] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:46] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:46] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:46] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:46] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:46] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
[14:39:46] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range
```

```r
# View the best-tuned parameters
print(xgb_tune$bestTune)
```

```
   nrounds max_depth eta gamma colsample_bytree min_child_weight subsample
18     100         3 0.1   0.1              0.7                1       0.7
```

```r
# Make predictions on the test set
y_pred_xgb <- predict(xgb_tune, X_test_matrix)

# Calculate R^2 on the test set
rss_xgb <- sum((y_test - y_pred_xgb)^2)
tss_xgb <- sum((y_test - mean(y_test))^2)
r_squared_xgb <- 1 - (rss_xgb / tss_xgb)

# Print the R^2 value
print(paste("R^2 for xgboost model:", r_squared_xgb))
```

```
[1] "R^2 for xgboost model: 0.412278123516019"
```

- Hint: Workflow: Boosting tree for Prediction is a good starting point.

0.1.6 Summary (30pts)

Your score for this question is largely determined by your final test performance.

Summarize the performance of different machine learning forecasters in the following format.

```r
# Extract CV R² from the Random Forest model
print("Random Forest Cross-Validation R² values:")
```

```
[1] "Random Forest Cross-Validation R² values:"
```

```r
print(rf_tune$results[, c("mtry", "RMSE", "Rsquared")])
```

```
  mtry      RMSE  Rsquared
1    2 0.1529140 0.5675632
2    4 0.1507468 0.5752517
3    6 0.1505042 0.5755052
4    8 0.1506499 0.5743596
5   10 0.1506132 0.5743616
```

```r
# Predict on the test set
rf_predictions <- predict(rf_tune, newdata = test_data)

# Calculate test R²
rf_test_r2 <- cor(rf_predictions, test_data$log_volume)^2
print(paste("Random Forest Test R²:", rf_test_r2))
```

```
[1] "Random Forest Test R²: 0.406313306447612"
```

```r
# Extract CV R² from the XGBoost model
print("XGBoost Cross-Validation R² values:")
```

```
[1] "XGBoost Cross-Validation R² values:"
```

```r
print(xgb_tune$results[, c("nrounds", "max_depth", "eta", "Rsquared")])
```

```
   nrounds max_depth  eta  Rsquared
1       10         3 0.01 0.5247232
4       10         3 0.01 0.5272204
13      10         3 0.10 0.5338168
16      10         3 0.10 0.5373229
7       10         5 0.01 0.5401759
10      10         5 0.01 0.5401221
19      10         5 0.10 0.5483963
```

```
22       10         5 0.10 0.5534076
2        50         3 0.01 0.5380642
5        50         3 0.01 0.5371196
14       50         3 0.10 0.5715226
17       50         3 0.10 0.5769648
8        50         5 0.01 0.5544675
11       50         5 0.01 0.5579554
20       50         5 0.10 0.5761152
23       50         5 0.10 0.5753642
3       100         3 0.01 0.5452237
6       100         3 0.01 0.5451336
15      100         3 0.10 0.5754122
18      100         3 0.10 0.5789053
9       100         5 0.01 0.5635220
12      100         5 0.01 0.5657382
21      100         5 0.10 0.5710687
24      100         5 0.10 0.5729876
```

```r
# Predict on the test set
xgb_predictions <- predict(xgb_tune, newdata = test_data)

# Calculate test R²
xgb_test_r2 <- cor(xgb_predictions, test_data$log_volume)^2
print(paste("XGBoost Test R²:", xgb_test_r2))
```

```
[1] "XGBoost Test R²: 0.418778628055829"
```

Method	CV R^2	Test R^2
Baseline	0.57	0.35
AR(5)	0.565	0.41
Random Forest	0.573	0.40
Boosting	0.52	0.41

0.2 ISL Exercise 12.6.13 (90 pts)

On the book website, www.statlearning.com, there is a gene expression data set (Ch12Ex13.csv) that consists of 40 tissue samples with measurements on 1,000 genes. The first 20 samples are from healthy patients, while the second 20 are from a diseased group.

(a) Load in the data using read.csv(). You will need to select header = F.

```
#load dataset
url <- "http://www.statlearning.com/s/Ch12Ex13.csv"
ch12Ex13 <- read_csv(url, col_names = paste("ID", 1:40))


# Check the first few rows of the data
head(data)
```

```
1 function (..., list = character(), package = NULL, lib.loc = NULL,
2     verbose = getOption("verbose"), envir = .GlobalEnv, overwrite = TRUE)
3 {
4     fileExt <- function(x) {
5         db <- grepl("\\\\.[^.]+\\\\.(gz|bz2|xz)$", x)
6         ans <- sub(".*\\\\.", "", x)
```

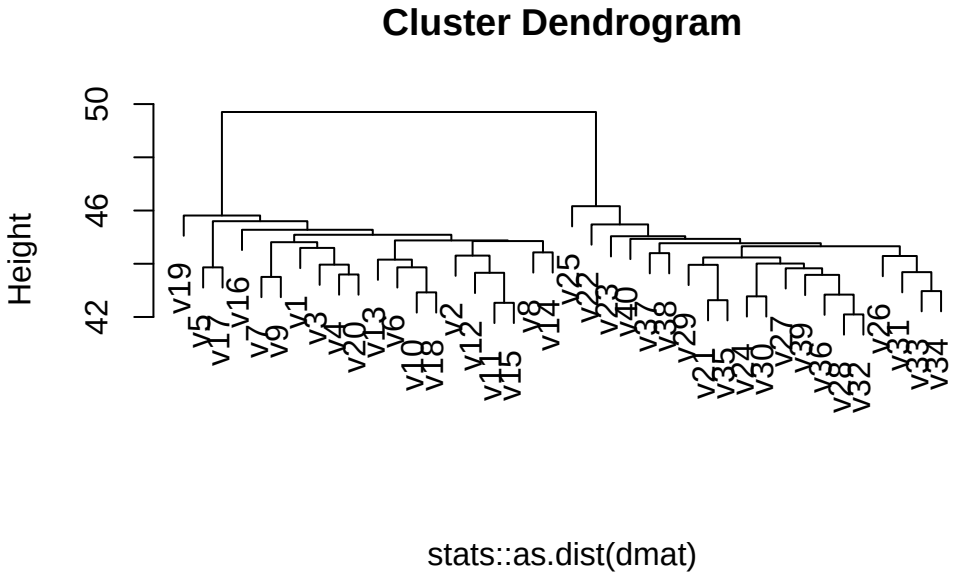
(b) Apply hierarchical clustering to the samples using correlation based distance, and plot the dendrogram.

```
#set up hierarchical clustering model (average linkage)

set.seed(123) #set seed for reproducability

data_transposed <- as.data.frame(t(ch12Ex13)) #transpose data

hc_spec <- hier_clust(
  # num_clusters = 3
  linkage_method = "average"
)

hc_fit <- hc_spec |>
  fit(~.,
      data = data_transposed
    )

hc_fit |>
  summary()
```

```
        Length Class      Mode
spec    4      hier_clust list
```

```
fit     7      hclust     list
elapsed 1      -none-     list
preproc 4      -none-     list
```

```
hc_fit$fit |> plot(sub = "")
```

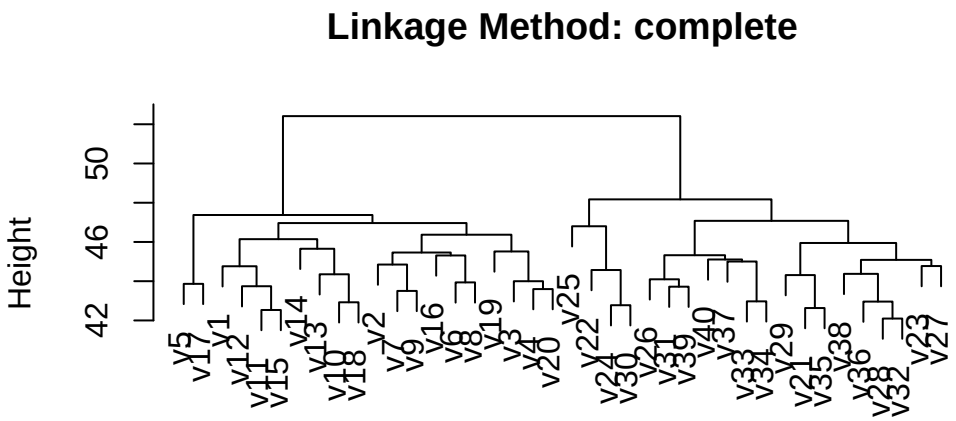


Do the genes separate the samples into the two groups?

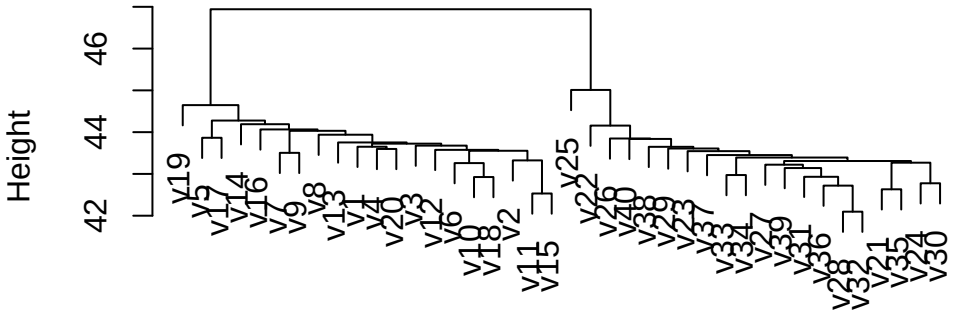
Yes we can see clearly that the samples are divided into two groups. Samples 1- 20 are in one cluster, while samples 21-40 are in another cluster.

```
#different linkage methods

set.seed(123) # set seed for reproducibility

# Define the list of linkage methods
linkage_methods <- c("complete", "single", "average", "ward.D", "centroid")

# Iterate through the linkage methods and plot the clustering results
par(mfrow = c(1, 1)) # Arrange the plots in a grid (2 rows, 3 columns)

for (method in linkage_methods) {
```

```
  hc_spec <- hier_clust(
    linkage_method = method
  )

  hc_fit <- hc_spec |>
    fit(~.,
        data = data_transposed
      )

  # Plot the dendrogram for the current linkage method
  hc_fit$fit |> plot(main = paste("Linkage Method:", method))
}
```

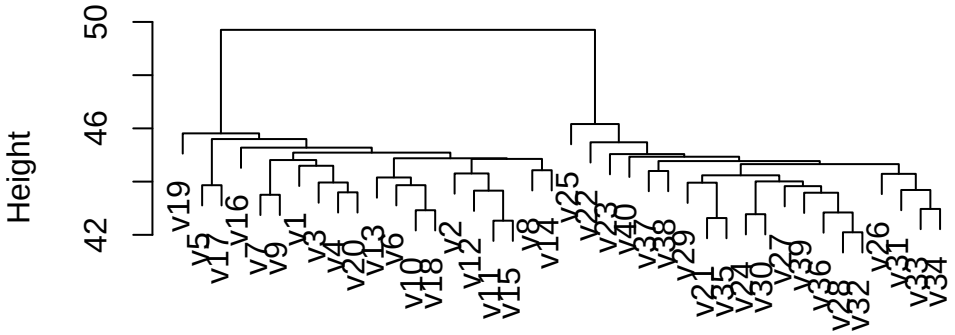


Linkage Method: single

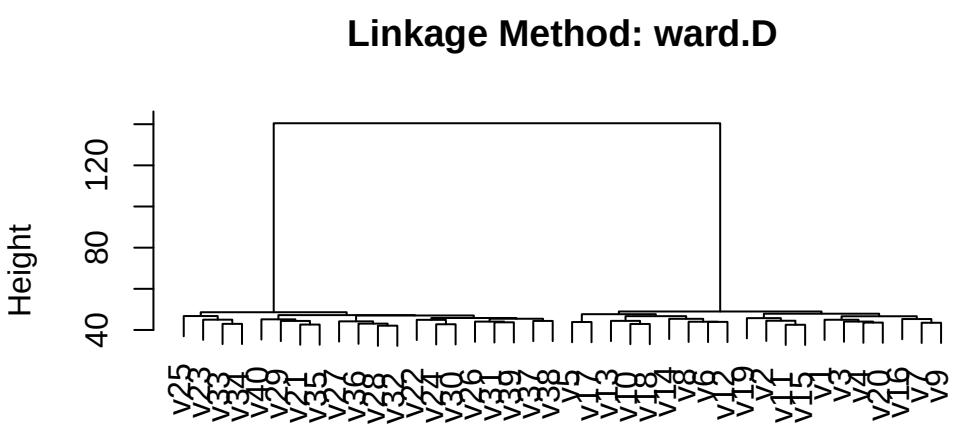


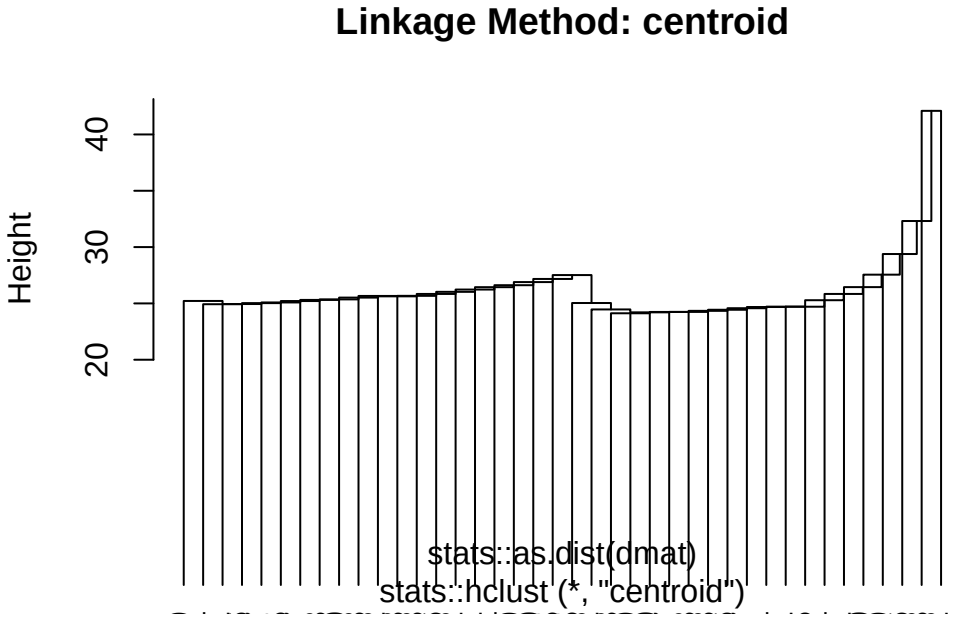
stats::as.dist(dmat)
stats::hclust (*, "single")

Linkage Method: average



stats::as.dist(dmat)
stats::hclust (*, "average")





Do your results depend on the type of linkage used?

It still appears that there are 2 distinct groups of some sort present every linkage type tested here. Some methods flip where each group is located on the dendrogram, but the samples themselves still appear to be in the original groups that the "average" method used.

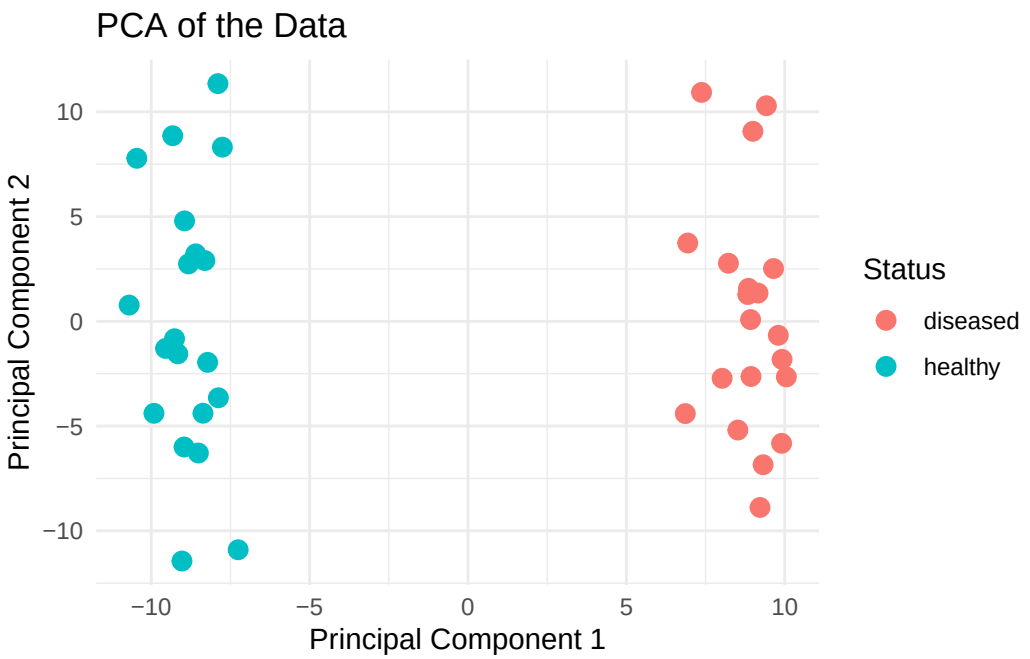
0.2.1 PCA and UMAP (30 pts)

Now we want to see the clusters from the genes using a PCA map and a UMAP. We can see there is a clear clustering of 2 groups from both the UMAP and the PCA graphs. This helps us confirm there is a distinct difference between these two groups we saw from the dendrogram.

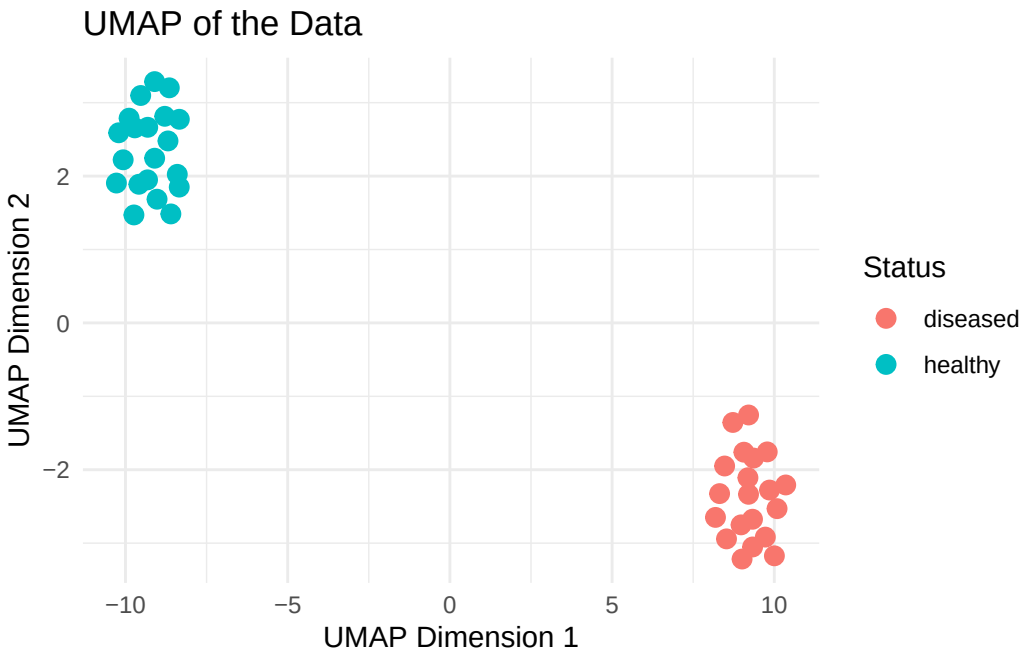
```
# Hierarchical clustering (average linkage)
set.seed(123) # set seed for reproducibility

# Add the "status" labels: first 20 "healthy", next 20 "diseased"
status <- c(rep("healthy", 20), rep("diseased", 20))

# Check the status labels
head(status)
```

```
[1] "healthy" "healthy" "healthy" "healthy" "healthy" "healthy"
```

```
# Perform PCA
pca_result <- prcomp(data_transposed, center = TRUE, scale. = TRUE)

# Create PCA data frame
pca_data <- data.frame(pca_result$x) # Extract PCA components
pca_data$Status <- factor(status) # Add "status" to the PCA data frame

# PCA plot with "status" coloring
ggplot(pca_data, aes(x = PC1, y = PC2, color = Status)) +
  geom_point(size = 3) +
  labs(title = "PCA of the Data", x = "Principal Component 1", y = "Principal Component 2") +
  theme_minimal()
```




```
# UMAP Plot

# Perform UMAP
umap_result <- umap(data_transposed)

# Create UMAP data frame
umap_data <- data.frame(umap_result$layout)

# Rename columns for clarity
colnames(umap_data) <- c("UMAP1", "UMAP2")  # Rename the UMAP dimensions

# Add "status" to the UMAP data frame
umap_data$Status <- factor(status)

# UMAP plot with "status" coloring
ggplot(umap_data, aes(x = UMAP1, y = UMAP2, color = Status)) +
  geom_point(size = 3) +
  labs(title = "UMAP of the Data", x = "UMAP Dimension 1", y = "UMAP Dimension 2") +
  theme_minimal()
```



0.2.2 12.6.13 (c) (30 pts)

Your collaborator wants to know which genes differ the most across the two groups. Suggest a way to answer this question, and apply it here.

What we can first do is cut our dendrogram to form two groups. These groups will correspond to the experimental groups we are interested in comparing.

We can perform a differential expression analysis (t-test) for each gene.

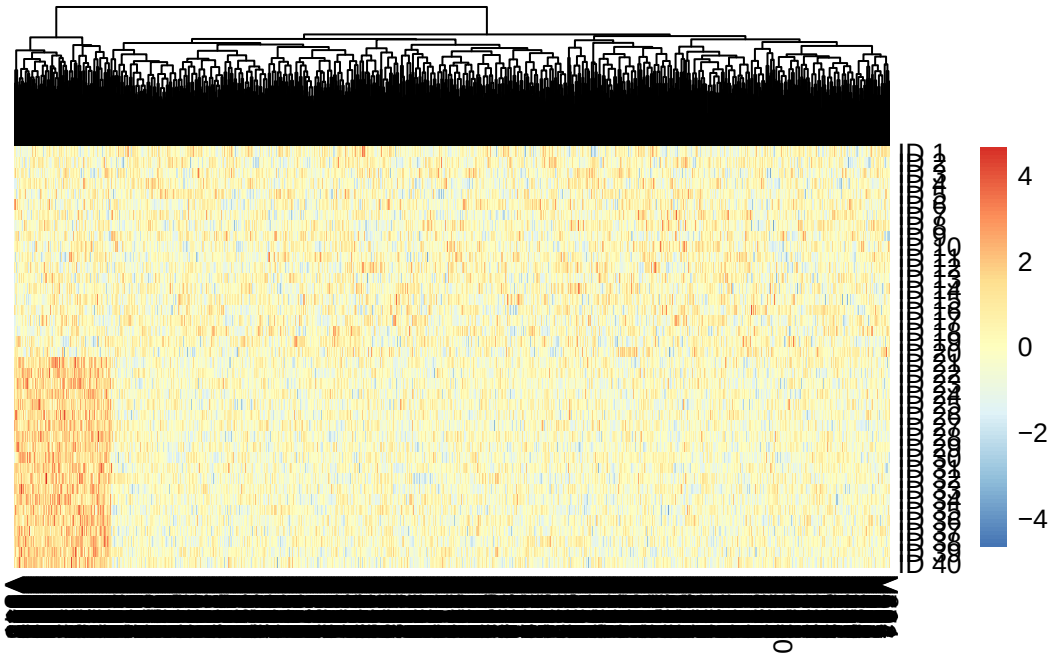
For each gene a t-test is performed comparing the expression values between the two groups. The t-test checks if there is a significant difference in gene expression between the two groups for each individual gene. The null hypothesis is that the mean expression level is the same in both groups, and the alternative hypothesis is that they are different. We then iterate over each row (gene) of the transposed data and run the t-test for each gene.

```
# Add status to the data (assuming `status` is already defined: "healthy" and "diseased")
# We assume `data_transposed` has genes as columns and samples as rows

# Create a data frame to hold the t-test results
t_test_results <- data.frame(Gene = colnames(data_transposed),
                             P_Value = NA,
                             Diff_Mean = NA)
```

```r
# Perform the t-test for each gene
for (gene in colnames(data_transposed)) {
  # Perform t-test comparing "healthy" vs. "diseased"
  t_test <- t.test(data_transposed[status == "healthy", gene],
                   data_transposed[status == "diseased", gene])

  # Store the p-value and the mean difference in the results
  t_test_results[t_test_results$Gene == gene, "P_Value"] <- t_test$p.value
  t_test_results[t_test_results$Gene == gene, "Diff_Mean"] <-
    mean(data_transposed[status == "diseased", gene]) -
    mean(data_transposed[status == "healthy", gene])
}

# Sort by p-value (smallest to largest) to identify the most significant genes
t_test_results_sorted <- t_test_results[order(t_test_results$P_Value), ]

# Display the top 10 most significant genes
head(t_test_results_sorted, 10)
```

```
    Gene      P_Value Diff_Mean
502 V502 1.530472e-12  2.544461
589 V589 6.511864e-12  2.343491
600 V600 1.003599e-11  2.747577
590 V590 1.123456e-10  2.205324
565 V565 1.765864e-10  2.470820
551 V551 1.195743e-09  2.342845
593 V593 1.350119e-09  2.402096
584 V584 2.132610e-09  2.601985
538 V538 2.548944e-09  2.391979
569 V569 2.830488e-09  1.976160
```

```r
library(pheatmap)

pheatmap(data_transposed, cluster_rows = FALSE, cluster_cols = T, scale ="row",
         annotiation_colors = list(status = c(disease ="orange", healthy = "black",
                                              color = colorRampPalette(c("navy", "white", "r
```


4
2
0
−2
−4
0